

Approccio agli esercizi delle tracce

- **Esercizi algebra relazionale:** 100% Jesus 🙏;
- **Esercizi creazione schema ER:** semplice ma fare attenzione ai vincoli d'integrità o eventuali scelte di progettazione (come le cardinalità) da motivare successivamente;
- **Esercizi di normalizzazione:** Trovare le dipendenze funzionali e creare delle tabelle partendo da quelle (verificare che le tuple vadano bene con la 2NF, ovvero che ogni attributo deve dipendere da tutta la chiave) e otterrai la 3NF. Nel caso di Boyce-Codd il determinante delle dipendenze deve essere sempre una chiave;
- **Traduzione in schema relazionale:** semplice ma attenzione alle chiavi esterne. Nel caso di relazione 1 a N, la chiave esterna va messa alla relazione dalla parte N (es. squadra formata da N giocatori, la chiave va messa a giocatore), nel caso di N a N va creata una tabella intermedia con chiave le chiavi delle due relazioni che sono connesse, non va aggiunta chiave esterna a nessuna delle due relazioni originali;
- **Esercizi di calcolo dei costi:** fare **tavola dei volumi** (concetto, tipo, volume), **tavola delle operazioni** (numeroOP, tipo (interattivo, batch), frequenza) e **tavola degli accessi** (operazione, concetto, accessi, tipo (lettura, scrittura)). Quest'ultima deve essere fatta **sia** per la situazione **con ridondanza** che quella **senza ridondanza**. Fare la somma dei costi di ogni operazione (per ogni operazione: $\sum(\text{lettura/scrittura} * \text{accessi}) * \text{frequenzaOperazione}$) e valutare il mantenimento della ridondanza. Per il calcolo degli accessi potrebbe essere necessario **ricavare dei valori medi** dai dati a disposizione;
- **Esercizi di ristrutturazione:** per ogni generalizzazione, valutare quale tipo di ristrutturazione (accorpamento del padre nei figli, accorpamento dei figli nel padre, entità separate) in base alla situazione. Il caso di **accorpamento dei figli nel padre** è utile quando si fanno **spesso operazioni che riguardano attributi sia dei figli che del padre**; è una soluzione che, a costo di **spazio sprecato per attributi nulli**, risulta **flessibile in caso di implementazione di sovrapposizione/esclusività in modo "intercambiabile"** (cambiare la proprietà non porterà complessità nella ristrutturazione) e risulta **più semplice accedere ai dati** poiché **non vengono svolte operazioni di join**. Il caso di **accorpamento del padre nei figli** è utile nel caso di **pochi attributi dell'entità padre** e vengono svolte **solo query specifiche ai singoli tipi di figlio**. Rende **difficile recuperare i dati esclusivi delle entità padre generiche** e risulta **meno flessibile nei casi di query con diversi tipi di figli** coinvolti. È valida come opzione nel caso in cui si vogliano **specificare dei vincoli sugli attributi in base a specifici casi** (da figlio a figlio ecc.).